

SecurityAI

Detailed Hackathon Execution Plan

Product: AI-powered security copilot SDK

Goal: ingest app security events, detect suspicious patterns, map them to MITRE ATT&CK, and display alerts in a clean dashboard.

Format: 4 remote contributors, one shared codebase, one live demo, one night.

This plan is designed for speed, clean division of labor, and a demo that looks far more ambitious than the code underneath. That is the hackathon tradition: product theater with enough engineering to avoid embarrassment.

1. Product Definition

The product is a developer-facing SDK called SecurityAI. A developer installs the SDK in their application and emits security events such as failed logins, successful logins, password resets, suspicious IP activity, and generic logs. Those events are sent to your backend, grouped into short timelines, analyzed by an LLM, mapped to MITRE ATT&CK techniques, and displayed on a dashboard.

Core user promise

A small team can connect the SDK in minutes and immediately get readable security alerts instead of raw logs. The system should answer: what happened, how severe is it, what MITRE technique does it resemble, and what should the developer do next?

Hackathon scope

- Support a simple JavaScript SDK.
- Accept events through one REST endpoint.
- Store events in SQLite or an in-memory store.
- Run AI analysis on event sequences.
- Display a dashboard with alerts, details, and MITRE tags.
- Provide one working demo flow that can be repeated live.

Not in scope

- Full log collection agents.
- Cloud provider integrations.
- Production-grade auth, billing, and multi-tenant isolation.
- A complete MITRE knowledge graph.
- Training a new ML model from scratch.

2. System Architecture

Layer	What it does	Owner
SDK	Captures security events from the developer app and posts them to the backend.	Member C
API Backend	Receives events, validates payloads, stores them, and exposes analysis endpoints.	Member A
AI Engine	Turns event sequences into threat labels, severity, explanation, and MITRE mapping.	Member A
Dashboard	Shows active alerts, event history, details, and natural-language summaries.	Member B
Demo Data	Produces realistic attack sequences and MITRE lookup data for the presentation.	Member D

Data flow

1. Developer calls SDK functions such as `security.auth.failedLogin()` or `security.log()`.
2. SDK normalizes the event and sends JSON to `POST /events`.
3. Backend stores the event and updates a short event timeline.
4. Analysis endpoint sends the timeline to the LLM with a strict JSON response format.
5. The LLM returns threat type, confidence, severity, explanation, and MITRE technique.
6. Dashboard fetches alerts and renders the result.

Key design choice

Keep detection and presentation separate. The backend can do simple rule checks, but the main intelligence should come from a structured LLM prompt that converts raw events into a readable incident report. That gives you a product that feels adaptive rather than hardcoded.

3. Detailed Workstreams

Team Member A - Backend and AI

- Define the event schema before anyone else writes code.
- Implement POST /events with validation, timestamp normalization, and project_id support.
- Implement GET /alerts and GET /events for the dashboard.
- Store events in SQLite or an in-memory list if time becomes tight.
- Create /analyze to build a small timeline and send it to Gemini.
- Return structured JSON with fields: attack, severity, confidence, mitre, reasoning, recommendation.
- Add a small MITRE mapping dictionary for fallback or post-processing.

Team Member B - Frontend Dashboard

- Build a clean landing page with product name, status indicators, and a live alert feed.
- Show total events, active threats, critical alerts, and last analyzed timestamp.
- Create a threat detail panel with severity, confidence, reasoning, and MITRE technique.
- Add a sample natural-language query box: 'What happened in the last hour?'
- Use mock data early, then connect to the live backend endpoints.
- Style the dashboard so the demo looks polished even if the backend is still being glued together.

Team Member C - SDK

- Create a JavaScript package with a simple constructor: new SecurityAI({ apiKey, projectId }).
- Implement security.log(), security.auth.failedLogin(), security.auth.successfulLogin(), and security.report().
- Normalize each helper into the same event payload format.
- Automatically attach client metadata such as app version, hostname, and timestamp.
- Keep the SDK thin and reliable. It should only format and forward events, not become a second backend.

Team Member D - Demo Data and MITRE Support

- Build 3 to 5 realistic attack sequences for the demo.
- Create a compact MITRE lookup file for common techniques.
- Write a demo app or script that generates the exact events needed to trigger alerts.
- Prepare screenshots, slide snippets, and example outputs.
- Own the presentation narrative and keep the demo reproducible.

4. Shared API Contract

The fastest way to break a remote team is to let everyone invent their own payload shape. Lock the contract first, then code against it.

Endpoint	Method	Purpose	Example
/events	POST	Receive normalized SDK events.	failed_login, successful_login, log
/alerts	GET	Return recent threats and scores.	High brute force alert
/analyze	POST	Analyze an event sequence with the LLM.	timeline -> JSON report
/events/latest	GET	Fetch recent events for the dashboard.	last 50 events

Canonical event payload

```
{
  "projectId": "demo-project",
  "event": "failed_login",
  "user": "admin",
  "ip": "192.168.1.5",
  "service": "auth-service",
  "timestamp": "2026-05-30T10:00:00Z",
  "metadata": {}
}
```

Response format from AI

```
{
  "attack": "Brute Force",
  "severity": "High",
  "confidence": 0.92,
  "mitre": "T1110",
  "reasoning": "Multiple failed logins followed by a successful login.",
  "recommendation": "Enable MFA and investigate the source IP."
}
```

5. Work Schedule for the Night

Time Window	Milestone	What must be true before moving on
0:00 - 0:30	Architecture lock	Event schema, endpoints, and demo goal are fixed.
0:30 - 2:30	Parallel build	SDK, backend, frontend, and demo data work in parallel.
2:30 - 3:30	First integration	SDK can hit /events and the backend stores data.
3:30 - 4:30	Dashboard hookup	Frontend reads live alerts and events.
4:30 - 5:30	AI prompt tuning	LLM output is structured and consistent.
5:30 - 6:30	MITRE mapping	Alerts include technique IDs and names.
6:30 - 7:30	Demo polishing	UI, sample data, and presentation are cleaned up.
7:30 - end	Rehearsal	The demo runs three times without breaking.

Hard rule

No one merges directly into the main branch without an integration check. For a remote team, merge conflicts are basically a sport. Keep one person responsible for final integration.

Suggested communication protocol

- One shared channel for blockers.
- One shared board with three columns: Todo, In Progress, Done.
- Short updates every 45 minutes.
- Any API change must be announced before implementation.

6. Demo Script

- 1 Open the demo app and show the SDK installation snippet.
- 2 Run the sample app or trigger a login sequence.
- 3 Generate multiple failed logins followed by a success.
- 4 Show the backend event stream receiving the events.
- 5 Open the dashboard and display the alert card.
- 6 Click the alert and reveal severity, confidence, reasoning, and MITRE technique.
- 7 Ask the natural-language summary question and show the AI explanation.
- 8 Close by explaining how the SDK could later plug into Slack, email, or real observability tools.

Fallback demo

If the live integration fails, switch to prerecorded events and a mocked API response. Nobody awards extra points for watching a live crash. The demo is what matters.

7. Risks and Mitigations

Risk	Mitigation
LLM output is inconsistent	Force JSON output and validate it before rendering.
SDK and backend disagree on payload shape	Define the schema first and publish it in a shared file.
Frontend has no live data	Use mocked endpoints until backend is stable.
Demo fails under time pressure	Keep one prerecorded scenario as a backup.

8. Final Execution Rule

Build the minimum path from SDK to alert card first. Only after that should anyone touch polish, extra features, or fancy charts. Pretty nonsense without a working loop is just a decorative failure.